SQL in Spark and Databricks

For data engineers and analysts, SQL is a fast path from raw data to trusted, scalable insight. In Spark and Databricks, it works alongside DataFrames, notebooks, and pipelines to support exploration, transformation, and production workflows.

FOUNDATIONS | SPARK | DATABRICKS

Why SQL Matters

Accessible

SQL is widely understood across analytics, engineering, finance, and operations. That shared language reduces handoff friction and makes collaboration easier.

Performant

In Spark, SQL benefits from the same optimiser and distributed execution engine as other APIs, so clear declarative logic can still scale.

BI-friendly

Most reporting and dashboard tools speak SQL naturally, which makes it the bridge between data platforms and business consumption.

In practice, SQL matters because it combines readability with operational value. Teams can prototype queries quickly, standardise transformations, and expose curated datasets to BI tools without translating logic into another language first.

DataFrame vs SQL

DataFrame	SQL
Programmatic and composable inside Python or Scala notebooks	Declarative and often easier to read for filtering, joining, and aggregating
Strong fit for dynamic logic, functions, loops, and reusable code patterns	Strong fit for analysis, reporting logic, and transformation steps that map cleanly to relational thinking
Often preferred by engineers building application-like pipelines	Often preferred by analysts and mixed technical teams working in notebooks and BI tools
Can feel verbose for simple queries	Can become harder to manage when logic is highly procedural

Under the hood, both approaches can produce similar execution plans in Spark. The key decision is usually readability, team skill set, and how dynamic the transformation logic needs to be.

Creating Temp Views

```
df.createOrReplaceTempView("table")
```

What it does

A temporary view gives a DataFrame a SQL name inside the current Spark session.

- ### Register once

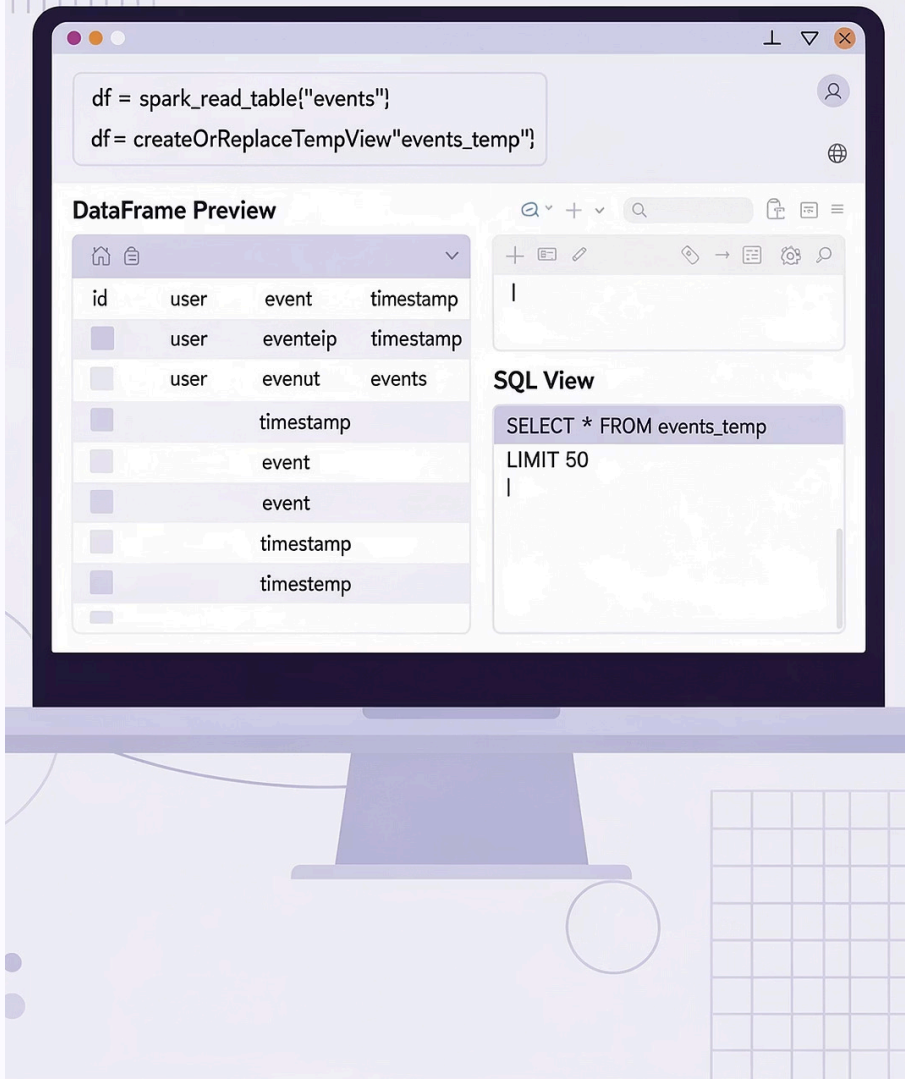
  Create the DataFrame in code, then expose it as a view for SQL queries.

- ### Session-scoped

  The view exists only for the active session unless you create a persistent table instead.

- ### Great for learning

  This is the easiest way to move between DataFrame logic and SQL in the same notebook.



```
1 冉牽海育= 非泯 {
2   Select id, name FROM users;
3 }
4
```

id	name
1	
Alice	
2	
Bob	

CORE PATTERN

Querying with SQL

```
SELECT * FROM table
```

Once a temporary view exists, you can query it with standard SQL. This is useful for inspection, validation, and building transformations step by step. A simple `SELECT *` is often the first move when you want to confirm the schema, preview records, or check whether an upstream DataFrame has been registered correctly.

01

Register the view

Turn a DataFrame into a SQL-accessible object.

02

Run a query

Filter, select, join, or aggregate using familiar SQL syntax.

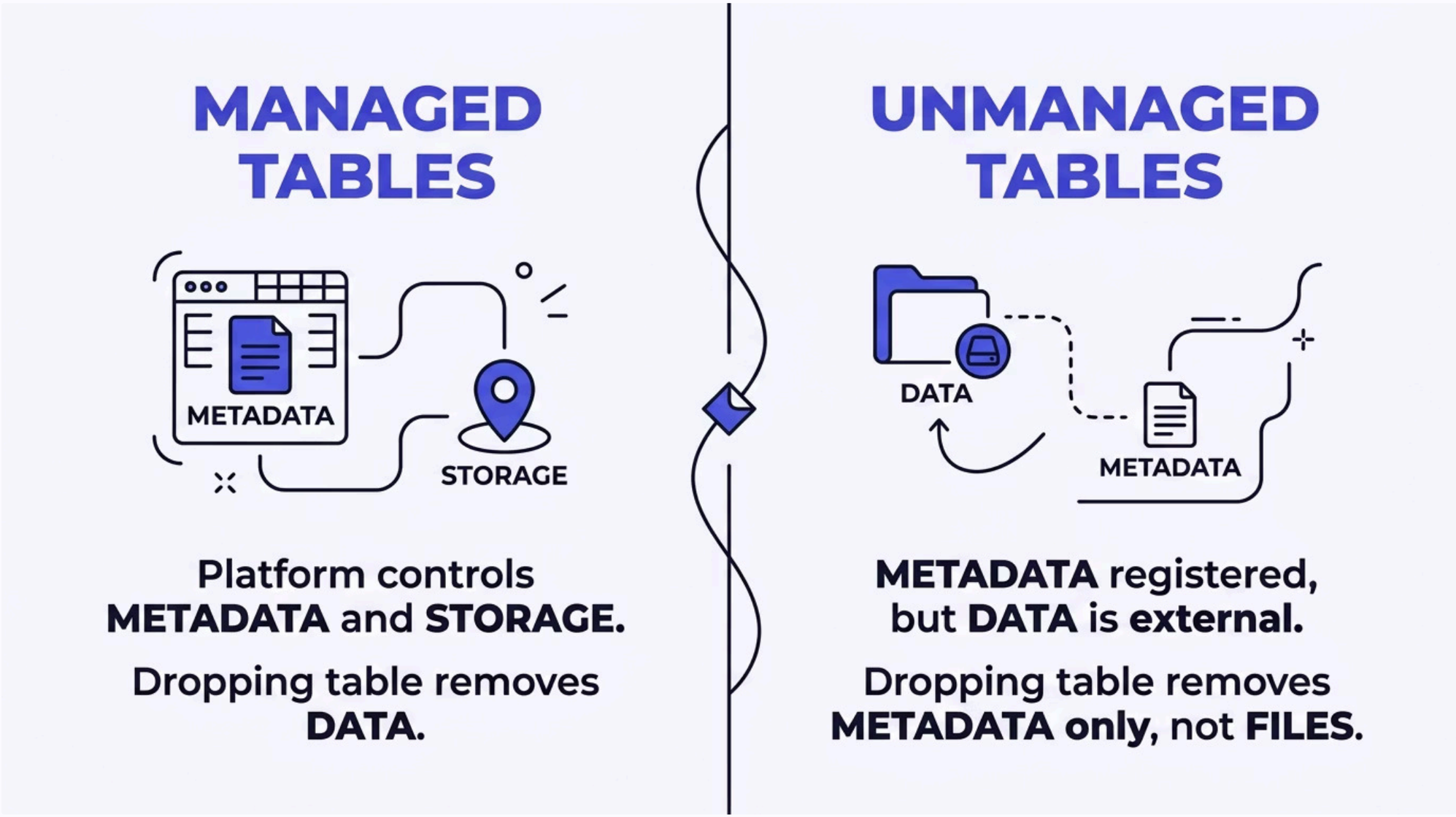
03

Validate output

Inspect the result before using it in further analysis or production logic.

STORAGE MODEL

Managed vs Unmanaged Tables



The key difference is storage ownership. Managed tables are controlled by the platform, which simplifies lifecycle management. Unmanaged, or external, tables point to data stored outside the default managed location. They are common when teams want tighter control over files, shared storage, or lakehouse architecture patterns.

CHAPTER 4 ORGANISATION

Database Concepts

```
CREATE DATABASE analytics;
USE analytics;
```

CREATE DATABASE

Defines a logical container for tables, views, and other objects. It helps teams organise assets by domain, project, or environment.

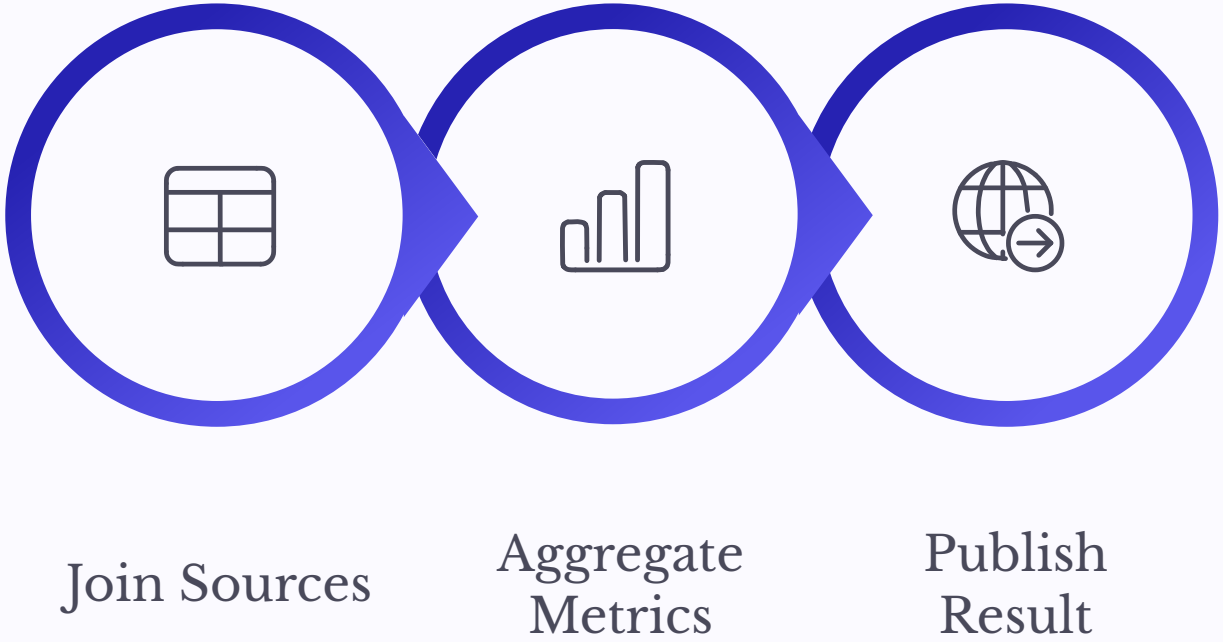
USE

Sets the active database so you can reference objects without repeating the full database name in every query.

Why it matters

Good database structure improves discoverability, governance, and day-to-day productivity in shared Databricks workspaces.

Joins & Aggregations



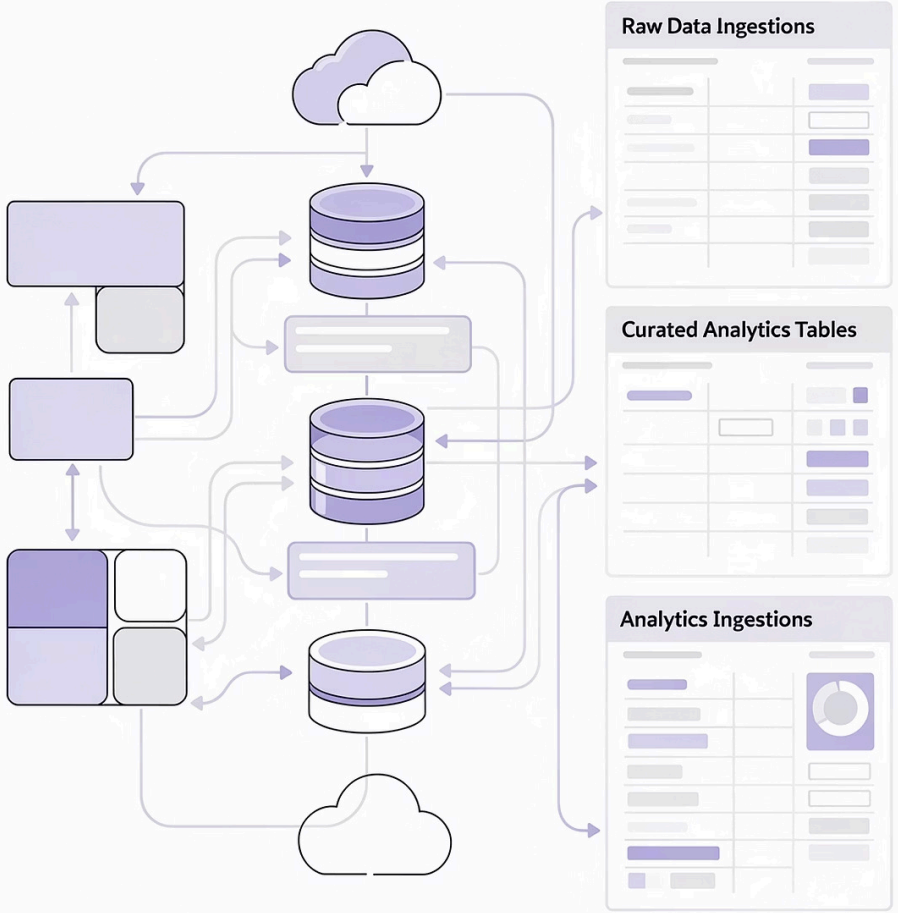
Joins and aggregations are where SQL becomes genuinely useful in production. Teams combine operational tables, enrich records with dimensions, and compute metrics such as revenue, customer counts, or daily activity. These patterns sit behind dashboards, KPI tables, and feature engineering datasets, so mastering them is essential for practical Spark SQL work.

DECISION GUIDE

When to Use What

Scenario	Use
Ad hoc analysis, validation, exploration, or BI-facing logic	SQL
Complex procedural logic, reusable functions, or application-style pipeline code	DataFrame
Mixed notebook workflow where engineers and analysts collaborate	Both
Transformations that benefit from readability and easy review	SQL
Dynamic logic generated from parameters or program control flow	DataFrame

The best teams are fluent in both. Rather than treating SQL and DataFrames as rivals, use each where it is strongest. In Spark, flexibility is an advantage: you can build with code, express logic in SQL, and still run on the same distributed engine.



NEXT CHAPTER

So far, we have covered the interface choices and the core objects that make SQL work in Spark and Databricks. The next step is to move beyond isolated queries and start designing transformations the way production data platforms do: layered, testable, and reusable. That means turning raw inputs into trusted intermediate models and business-ready outputs.

Ingest	Transform
Land raw data	Apply SQL logic
Publish	
Serve reliable tables	